Polyglot Proxy

The Intelligent Edge Router for Agentic AI Workflows.

PROJECT: AWS HACKATHON TRACK: AGENT EXPERIENCE MODEL: STRANDS DECIDER 2B

1. Executive Summary

Polyglot Proxy is a high-speed, cost-saving middleware proxy designed to sit between user-facing applications and generative Large Language Models (LLMs). By leveraging the **Strands Decider 2B** model as an initial decision gate, the proxy evaluates incoming prompts in approximately 115ms. It intelligently routes simple tasks to cost-effective models (e.g., Claude 3 Haiku) and reserves frontier models (e.g., Claude 3.5 Sonnet) exclusively for complex reasoning tasks.

2. Problem Statement

As developers deploy autonomous AI agents in production, inference costs and latency become critical bottlenecks. Sending straightforward queries to massive frontier models wastes expensive compute resources. Furthermore, relying on large LLMs to verify code safety, check parameter grounding, or prune context from Vector Databases introduces unacceptable multi-second latency delays, degrading the overall user experience and inflating cloud billing.

3. Key Value Propositions

> **Cost Reduction:** Up to 70% savings on Amazon Bedrock API costs by dynamically routing simple queries to smaller, more efficient models rather than defaulting to frontier models.

> **Latency Optimization:** Enables real-time tool-call verification and prompt safety checks using local inference (~115ms), completely bypassing the typical 2-3 second LLM round-trip delay.

> **Context Pruning (RAG):** Scores and filters irrelevant context chunks retrieved from vector databases before LLM injection, mitigating prompt bloat and reducing hallucination risks.

> **Drop-in Reusability:** Packaged as a standalone Python module (`IntelligentRouter`) that any developer can seamlessly integrate into their existing AI workflows.

4. System Architecture Flow

The routing engine intercepts requests at the edge before dispatching to Amazon Bedrock.



5. Feature Requirements

- **FastAPI Gateway:** Must provide a REST endpoint for easy integration with existing Web UI applications, dashboards, or Chrome Extensions.
- **Local Inference Engine:** Hosts Strands Decider 2B locally (via PyTorch) to ensure routing decisions execute strictly under 150ms.
- **#model-routing Logic:** Classifies user prompt complexity via binary output. Routes to frontier models on `true` and compact models on `false`.
- **#guardrails Logic:** Intervenes before tool execution to verify that generated parameters are grounded in the user's initial state.

6. Success Metrics (KPIs)

< 150ms	40%+	> 90%
Decision Latency	**Cost Efficiency**	**Routing Accuracy**
Target execution time for local Decider 2B classification.	Minimum simulated Bedrock API savings on mixed workloads.	Correct identification of complex vs. simple queries.

7. Technical Stack Specification

Component	Technology	Rationale
API Gateway Layer	FastAPI (Python), Uvicorn	High concurrency, native async operations, easy Pydantic schema validation.
Decision Engine	Strands Decider 2B, PyTorch	Mandatory for hackathon; provides sub-150ms binary text classification.
Generative Backend	Amazon Bedrock (`boto3`)	Enterprise-grade managed access to Claude 3 Haiku and Sonnet models.
Client Dashboard UI	Streamlit (Python)	Rapid deployment for real-time visualization of cost savings and metrics.
Browser Extension	JavaScript (Manifest V3)	Allows interception of native web prompt text areas prior to execution.
Analytics & Logging	PostgreSQL (Supabase)	Relational tracking of user token consumption, latencies, and routing logs.

8. Development Roadmap

Phase 1: Core Engine Integration

Build the FastAPI asynchronous proxy. Successfully load Strands Decider 2B via Hugging Face Transformers and establish secure API connections to Amazon Bedrock via AWS IAM roles.

Phase 2: Use Cases & Analytics

Implement the exact `#model-routing` prompt structures. Connect the backend to PostgreSQL to log request latencies, input token sizes, and simulated billing parameters.

Phase 3: Front-End Visualization

Develop the Streamlit web dashboard. Create interactive UI charts that display live token usage reductions and split-testing metrics to prove real-time financial savings.

Phase 4: Hackathon Packaging

Refine the `IntelligentRouter` into a standalone, reusable Python class. Record the required 3-minute demonstration video and draft the AWS Builder Center submission article featuring required tags.